IASA Masterclass · Report out

AIForge — our own AI developer

You give it a ticket. It gives back a tested pull request.
It runs on machines we own. Nothing leaves the building.

Roles only — no names, no customer detail.

The problem we had

Three facts about where we work (defence sector, government customer)

IASA tool
Business Model Canvas

1

Too many small tickets

Our best engineers spend the day on small, routine changes. The hard work waits.

2

Same team, more output

The plan keeps growing. We have to deliver more without growing the team.

3

Cloud AI is not allowed

No code or data may go to an outside service. Ever. So the usual AI tools are out.

In short: we needed AI help that runs inside our own walls — so we built it.

What AIForge is

One input, one output — and nobody has to watch in between

A ticket

Written in plain words, like you would ask a colleague

AIForge

A small team of AI agents on our own GPUs plans, writes, tests and reviews the change

A pull request

A tested, reviewed code change, ready for a person to read and merge

Two ways to use it

Chat — talk to it; it reads the code, runs commands, edits files.
Ticket — file it and walk away; a pull request appears.

Words used in this deck

Pull request (PR) — a proposed code change a person reviews.
Agent — an AI worker with one small job.

In short: a person only touches two moments — writing the ticket and reading the pull request.

Who it is for

Three people, described by role

IASA tool
Persona Card

Main user

Delivery engineer

“I know how to do this. I do not have the hours.”

Happy when: Small tickets are off their desk; a PR is waiting to read.

Learner

Junior developer

“I want to learn, not sit blocked.”

Happy when: The PR shows them the right pattern in our own code.

Decision maker

Engineering lead

“Show me it works before I plan around it.”

Happy when: One honest number lands each month.

In short: it is built for the delivery engineer. If their day does not change, nothing else matters.

Before and after

The same small ticket, today and with AIForge

IASA tool
Customer Journey

	Ticket raised	Picked up	Code written	Tested and reviewed	Merged
Today	sits in the backlog	waits for a senior	squeezed between other work	rushed at the end	days later
With AIForge	picked up as written	starts at once	agents write it in a copy of the code	tests first, second AI reviews	PR ready to read

In short: the cost was never the typing — it was the waiting. AIForge removes the wait.

How one ticket flows

Seven AI agents, each with one small job, in order

1	2	3	4	5	6	7
Enhancer	**Architect**	**Senior dev**	**Developer**	**Tester**	**Reviewer**	**Reconcile**
makes the ticket clear	splits the work, writes a shared plan	sets the shape of the code	writes each part	writes and runs tests	a second AI checks the work	joins it all and builds until it passes

Why split it up? Our local AI models can only hold about 3–5 files at once. Small jobs with only the context they need fit; one big job does not.

In short: one shared plan first, tests before code, and nothing is merged unless it builds.

What you see vs what runs behind

IASA tool
Service Blueprint

Everything below the line runs without anyone watching

	1. Ask	2. Work	3. Check	4. Hand back
You see	You write the ticket	A run in progress (you can leave)	Test results	A pull request with an honest verdict
	line of visibility			
Behind the scenes	Ticket is cleaned up and routed	Plan, shared spec, code written	Tests, second-AI review, build	Verdict written, lessons saved

In short: for the person it is two moments: file it, then read it. The design work is all below the line.

Three things it must get right

Break any one and the tool is not worth using

IASA tool
ASR Cards

1 HIGH

Do the job on its own

Ticket in, pull request out — nobody in the middle.

Proof: the first full run went from ticket to PR with no help.

2 HIGH

Make the files fit together

Change five files and they still work as one: imports connect, calls match, it builds.

Proof: 0 working linked lines → 53 once we added a shared plan.

3 MED-HIGH

Stay on our machines

Runs on hardware we own. No cloud. Works with the internet off.

Proof: zero runs have sent data outside our network.

In short: these three rules shaped every design choice.

Why we built our own

We compared four ways to get an AI developer

IASA tool
ADR

	Ready-made agent	One big model	Cloud AI	AIForge
Code stays inside	Yes	Yes	No	Yes
Multi-file change still builds	No	No	Yes	Yes
Checked by a second AI	No	No	No	Yes
Runs on our own machines	No	Yes	No	Yes

The cloud AI works best on paper — but it sends our code outside, which is not allowed. The ready-made agent broke on our model server and could not build Java.

In short: only AIForge passes every row — and our tests proved it.

What the numbers say

Measured on our own machines, not estimated

IASA tool
QATT Card

0 → 53

linked lines of working code in a multi-file ticket, once work was split with a shared plan

3–5

files one ticket can change cleanly today. Target: 8–10 after the GPU upgrade

0

runs where any data left our network

In short: splitting the work beat waiting for a bigger model.

Where we are today

Goal: make AIForge the normal way small work gets done

IASA tool
OKR Card

What we measure	Target	Today
Tickets that end in a merged pull request	**3 in 4**	**Not measured yet**
Files one ticket can change cleanly	**8 to 10**	**3 to 5**
Tickets filed by people other than the leads	**Half**	**Almost none**
People who can run and fix the platform	**2**	**1**
Runs where data left our network	**Zero**	**Zero**

Green = on target. **Orange** = still to close.

In short: the tool works; now we must measure how often it succeeds.

Demo: what you will see

Follow these five steps on screen

1	**Write a ticket**	A small change, in plain words — the way you would ask a colleague.
2	**Start the run**	AIForge picks it up. No setup, nobody needs to watch.
3	**Watch the agents**	The progress panel shows each step: plan, write, test, review.
4	**See the checks**	Tests run and a second AI reviews. If it fails, it says so in the first line.
5	**Read the pull request**	A finished change with an honest verdict, ready to merge.

Watch for

- Nothing leaves the machine
- Tests before code
- A second AI checks
- Honest verdict at the top

In short: you only act at step 1 and step 5. Steps 2–4 run on their own.

What is next

The tool works. What is left is people, not code.

Next steps

- Publish one number a month: work shipped vs problems caused
- Hand the model service to IT ops, with someone on call
- Train a second operator
- Test bigger local models after the GPU upgrade

What I learned

- The limit was the design, not the model
- The IASA canvases made the argument checkable — one page, not a long thread
- A tool one person can run is not yet a tool the company owns

One rule for the whole year: nothing moves work off our own hardware.